

UNIVERSIDAD ESPÍRITU SANTO

NOMBRE DE LA ASIGNATURA:

ESTRUCTURA DE DATOS

TEMA:

PROYECTO: BLACKBOARD

AUTORES:

KAREL LÁZARO GONZÁLEZ RUÍZ

JUSTIN AARÓN SOLEDISPA DUEÑAS

JUAN DIEGO SOTOMAYOR JIMÉNEZ

DOCENTE:

MARISOL VILLACRES

GUAYAQUIL-ECUADOR

2026

1. Organización modular del sistema

Una de las primeras decisiones fue estructurar el proyecto bajo un enfoque claramente modular. Esto significa que cada clase cumple una responsabilidad específica y no invade el espacio funcional de otra.

Las clases del modelo (Estudiante, Actividad, Entrega, Cálculo) representan las entidades principales del dominio. Por ejemplo, un estudiante almacena información personal; una actividad contiene fecha límite y tipo; una entrega almacena nota y fecha; un cálculo representa una fórmula evaluable por tipo de tarea. Por otro lado, la lógica de coordinación está distribuida en clases específicas:

- CargadorDatos: se encarga exclusivamente de leer archivos CSV y construir las estructuras internas del sistema.
- GestorConsultas y ServicioConsultas: ejecutan las consultas sobre ListaCompuesta<Actividad,Entrega> y ListaCompuesta<Estudiante,Entrega>
- GestorCalculos: ejecuta las fórmulas asociadas a las calificaciones que son evaluadas en EvaluadorCalculos
- GestorReportes: asegura el funcionamiento correcto del reporte de calificaciones
- App, VistaBienvenida, ExportadorReporte, FXListUtils, ModeloDatosFx, ModuloCrearActividad, ModuloConsulta y ModuloReporte: conforman la capa gráfica en JavaFX.
- SistemaConsola: organiza la interacción con el usuario, mostrando menús y recibiendo entradas.

2. Uso de genéricos

El uso de tipos genéricos fue una decisión estructural clave. Implementamos estructuras como ListaCompuesta<E, F> y NodoCompuesto<E, F> para permitir reutilización sin duplicación de código. La lista secundaria, con el fin de mantener la abstracción sin sacrificar complejidad en las operaciones es ListaCompuesta<F, Object> Gracias a los genéricos, la misma estructura puede representar diferentes relaciones: Estudiante/Actividad con sus entregas, o cálculos asociado a distintos elementos. Sin genéricos, habríamos tenido que crear múltiples versiones de listas para cada tipo de entidad, lo que generaría redundancia y mayor probabilidad de errores.

3. Uso estratégico de múltiples comparadores

Uno de los aspectos más relevantes del proyecto fue el uso extensivo de comparadores con Comparator<E>. La razón de tener varios comparadores es la flexibilidad. El sistema puede reutilizar la misma lógica de búsqueda u ordenamiento cambiando únicamente el criterio. Esto evita código repetitivo y mantiene el diseño limpio. Se implementaron comparadores para:

- Fecha límite.
- Calificación.
- Actividad específica.
- Estado de evaluación.
- Contenido o características particulares.

Cada comparador encapsula una regla concreta. En lugar de incluir múltiples condiciones dentro de los métodos principales, se delega la lógica de comparación a clases especializadas. Además, si surge un nuevo criterio de ordenamiento en el futuro, se puede implementar un nuevo comparador sin modificar las estructuras centrales del sistema.

4. Evaluación de expresiones en notación posfija mediante pila

Para la evaluación de fórmulas utilizamos una pila implementada con ArrayDeque. La notación posfija de los calculos requiere almacenar operandos y, al encontrar un operador, aplicar la operación a los dos últimos valores ingresados. Separar esta lógica en EvaluadorCalculos permitió mantener la coherencia estructural y aislar la lógica matemática del resto del sistema.

5. Manejo de archivos y persistencia

El sistema permite cargar datos desde archivos y generar reportes en formato de texto. Esta funcionalidad se implementó de forma separada en el Cargador. El manejo de archivos incluye:

- Carga estructurada de datos.
- Validación básica.
- Escritura de reportes con formato claro.

Separar la persistencia del resto del sistema evita que las entidades dependan de detalles externos, manteniendo el principio de responsabilidad única.

6. Presentación dual: Consola e Interfaz Gráfica

El proyecto incluye una la incorporación de una arquitectura dual:

- Modo consola
- Modo gráfico (JavaFX).

Ambos entornos reutilizan exactamente las mismas estructuras y lógica de negocio. La interfaz gráfica no contiene reglas adicionales, demostrando que el núcleo del sistema fue diseñado de manera independiente de la capa de presentación.

7. Generación de reportes dinámicos

El sistema permite:

- Selección múltiple de actividades.
- Selección múltiple de cálculos.
- Generación de tabla dinámica en consola.
- Exportación en formato TXT.
- Exportación en formato CSV.
- Generación de tabla dinámica en interfaz gráfica.

La tabla en JavaFX se construye dinámicamente según lo seleccionado por el usuario, creando columnas en tiempo de ejecución.

8. Fuentes consultadas y uso de IA

El desarrollo del proyecto se basó principalmente en el material proporcionado en clase, especialmente en los temas relacionados con:

- Tipos Abstractos de Datos.
- Uso de genéricos.
- Comparadores.
- Pilas y sus aplicaciones prácticas.

Adicionalmente, se consultó documentación oficial de Java para verificar detalles técnicos sobre:

- Implementación correcta de Comparator.
- Uso de Deque y ArrayDeque.
- Manejo de archivos con BufferedWriter y FileWriter.

La inteligencia artificial se utilizó como herramienta de apoyo para:

- Resolver dudas específicas de implementación.
- Construcción de interfaces con JavaFX.
- Validar decisiones de diseño.
- Mejorar la organización del código.
- Revisar coherencia de los datos.

Su uso fue complementario y enfocado en aclarar conceptos o confirmar buenas prácticas, no en reemplazar el desarrollo propio del proyecto

Diagrama de clases - Blackboard - Estructura de datos

Karel González, Justin Soledispa, Juan Diego Sotomayor | February 20, 2026

